Contents

Chapter 17: Agent Context Management

As coding agents become more capable, one of the most important engineering problems is no longer simply:

> **How do we make the model smarter?**

It is:

> **How do we give the model the right information at the right time?**

A software repository may contain hundreds of thousands or millions of tokens of potentially relevant information. The model, however, has finite context, finite attention, and finite ability to distinguish signal from noise.

This creates a new engineering discipline:

> **Context engineering is the systematic construction, management, compression, and delivery of information to an AI agent so that it can make reliable decisions.**

The distinction between **model capability** and **context quality** is fundamental.

A highly capable model with poor context can perform badly.

A somewhat weaker model with carefully constructed context can perform surprisingly well.

The agent therefore has two fundamental resources:

$$\boxed{\text{Reasoning capability} + \text{Relevant context}}$$

Chapter 16 showed how specifications constrain the space of acceptable solutions.

Chapter 17 focuses on the other side of the equation:

> **What information does the agent need in order to reason correctly about that specification?**

---

1. The Context Problem

Consider a repository:

```
my-service/
+-- src/
|   +-- api/
|   +-- auth/
|   +-- database/
|   +-- retrieval/
|   +-- models/
|   `-- services/
+-- tests/
+-- migrations/
```

```
+-- docs/
+-- scripts/
+-- deployment/
+-- config/
+-- generated/
+-- .github/
`-- README.md
```

Suppose the task is:

> Fix the authentication bug in the document retrieval endpoint.

Potentially relevant information includes:

```
src/api/retrieval.py
src/auth/
src/models/user.py
src/database/
tests/test_auth.py
tests/test_retrieval.py
configuration
API documentation
security documentation
```

But much of the repository may be irrelevant:

```
frontend/
deployment/
generated/
unrelated services/
historical documentation/
```

The naive strategy is:

```
Repository
    ↓
Everything
    ↓
LLM
```

The better strategy is:

```
Repository
    ↓
Context selection
    ↓
Relevant information
    ↓
LLM
```

This sounds simple.

It is not.

The agent must determine:

1. What information is relevant?
2. What information is authoritative?
3. What information can be summarized?
4. What information must be preserved verbatim?
5. What information can be discarded?
6. What information should be retrieved later?
7. How much context should be allocated to each category?

These are engineering decisions.

---

2. Context Is Not the Same as State

One of the most important distinctions in agent architecture is:

$$\boxed{\text{Context} \neq \text{State}}$$

State describes what is true about the system or task.

Context is the information currently presented to the model.

For example:

```
State:
- OAuth integration has been implemented.
- 3 tests are failing.
- The redirect URL is not preserved.
```

The model's current context might contain:

```
Task description
+
relevant source files
+
test failures
+
recent edits
+
architectural constraints
```

The state may persist across many context windows.

The context is a **projection of that state** into the model's current working window.

Formally:

$$C_t = P(S_t, H_t, R_t)$$

where:

- S_t = task/system state
- H_t = interaction history
- R_t = retrieved information
- P = context-construction policy

This distinction becomes essential for long-running agents.

The agent should not attempt to preserve everything.

It should preserve **state** and reconstruct **context** as needed.

---

3. Context as a Limited Budget

Context is a finite resource.

Suppose the model has a context capacity:

$$B$$

and the context contains several categories:

$$C = C_{\text{system}} + C_{\text{task}} + C_{\text{state}} + C_{\text{repository}} + C_{\text{tools}} + C_{\text{history}} + C_{\text{feedback}}$$

Then:

$$|C| \leq B$$

The problem becomes an allocation problem.

For example:

```
Context budget
+-- system instructions       10%
+-- task specification         5%
+-- repository context        30%
+-- relevant code             25%
+-- tests                     15%
+-- tool results              10%
`-- working state              5%
```

These percentages are illustrative, not universal.

The important idea is:

> **Context should be budgeted deliberately.**

If 80% of the context is consumed by irrelevant repository files, there may be insufficient room for the actual problem.

---

4. Context Selection

The first major operation is **selection**.

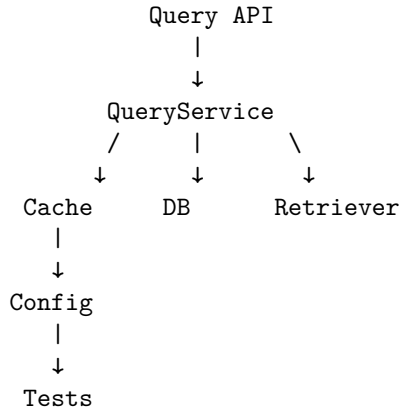
Suppose the task is:

> Fix the caching bug in `QueryService`.

A repository-aware agent might perform:

```
1. Find QueryService
2. Inspect its dependencies
3. Find callers
4. Find related tests
5. Find cache configuration
6. Search for previous cache-related fixes
```

This produces a dependency neighborhood:



The agent does not need the entire repository.

It needs the **relevant subgraph**.

This suggests a useful abstraction:

$$R_t = \text{RelevantSubgraph}(G, task)$$

where G is the repository's conceptual dependency graph.

The quality of context selection depends on how well the agent identifies this subgraph.

---

5. Repository Maps

A **repository map** is a compressed structural representation of a codebase.

Instead of giving the agent thousands of files, provide something like:

```
src/
+-- api/
|   +-- query.py          # HTTP query endpoint
|   `-- documents.py      # document management API
+-- services/
|   +-- query.py          # query orchestration
|   +-- retrieval.py      # hybrid retrieval
|   `-- embedding.py      # embedding generation
+-- auth/
|   +-- middleware.py     # authentication
|   `-- permissions.py    # authorization
`-- models/
    +-- document.py
    `-- user.py

tests/
+-- test_query.py
+-- test_retrieval.py
`-- test_permissions.py
```

This gives the model a **map before requiring it to explore the territory**.

A repository map can include:

- directory structure
- important modules
- public interfaces
- dependency relationships
- test locations
- configuration locations
- architectural boundaries

It functions somewhat like a symbol table or architectural index.

---

6. Context Selection as Retrieval

Context management is closely related to retrieval.

The agent can formulate a query:

> "Where is authorization enforced for document retrieval?"

and retrieve:

```
src/auth/permissions.py
src/api/query.py
tests/test_permissions.py
docs/security.md
```

This creates:

```
Task
 ↓
Context query
 ↓
Retrieval
 ↓
Relevant artifacts
 ↓
Context construction
 ↓
LLM
```

The retrieval mechanism can be:

- lexical search
- semantic search
- symbol search
- dependency analysis
- AST analysis
- repository graph traversal
- metadata filtering
- hybrid retrieval

This is why agent context management and RAG are closely related.

Traditional RAG retrieves **documents**.

Coding-agent context management retrieves **software artifacts and state**.

---

7. Context Selection Is More Than Retrieval

Pure retrieval is insufficient.

Suppose a task requires changing an API.

The most relevant source file may be:

```
src/api/query.py
```

But the agent may also need:

```
API contract
authentication requirements
database schema
related tests
architecture decision
deployment constraints
```

Some of these may not be lexically similar to the task.

Therefore:

$$\text{Useful Context} \neq \text{Top-k Search Results}$$

A better model is:

$$C_t = R_{\text{retrieval}} + R_{\text{structure}} + R_{\text{state}} + R_{\text{constraints}} + R_{\text{verification}}$$

The agent needs both **semantic relevance** and **structural relevance**.

---

8. Context Compression

Even relevant context may be too large.

Suppose the agent reads a 2,000-line module.

Only 100 lines may matter for the current task.

The harness can compress the remaining information.

For example:

```
Original:
2,000 lines

Compressed:
- module purpose
- public interfaces
- important classes
- invariants
- dependencies
- relevant functions
- known failure modes
```

Compression can be represented as:

$$C' = \text{Compress}(C)$$

with:

$$|C'| \ll |C|$$

while attempting to preserve task-relevant information:

$$I(C'; task) \approx I(C; task)$$

The problem is that compression is lossy.

If the compressor removes a subtle invariant, the agent may make an incorrect change.

Therefore, context compression should be **task-aware**.

---

9. Summaries

Summaries are one form of context compression.

A useful coding-agent summary might look like:

```
Repository state:

Task:
Add rate limiting to the query API.

Architecture:
FastAPI → QueryService → Retriever.

Current implementation:
- Authentication occurs in middleware.
- QueryService has no rate limiting.
- Redis is already available.

Constraints:
- Per-user rate limits.
- Must not affect internal service calls.
- Return HTTP 429 when exceeded.

Tests:
- tests/test_query.py
```

```
- tests/test_auth.py

Current progress:
- Middleware implementation complete.
- 2 tests failing.
```

This is far more useful than replaying 50 previous tool calls.

But summaries should preserve **decisions, assumptions, constraints, unresolved issues, and state transitions**.

A bad summary:

> "Worked on rate limiting. Some tests failed."

A good summary:

> "Rate limiting is implemented in API middleware using Redis. Internal service calls bypass the middleware. The remaining failure is `test_rate_limit_resets_after_window`, which indicates the Redis expiration is not being set correctly."

The second preserves actionable state.

---

10. Scratchpads

Agents also need temporary working memory.

A **scratchpad** contains information useful for the current reasoning process but not necessarily part of the permanent task state.

For example:

```
Current hypothesis:
Cache invalidation occurs before transaction commit.

Evidence:
- QueryService invalidates cache at line 142.
- Database commit occurs in Repository.save().
- Failing test observes stale data.

Next action:
Inspect transaction boundaries.
```

The scratchpad is different from:

```
Permanent architecture decision:
PostgreSQL is the source of truth.
```

This distinction is useful:

```
Persistent state
      |
      +-- architecture
      +-- decisions
      +-- constraints
      `-- progress

Ephemeral state
      |
      +-- hypotheses
      +-- intermediate reasoning
      `-- next-action ideas
```

The agent needs both, but they should not be treated as identical.

---

11. Documentation as Context

Documentation is often one of the highest-value sources of context.

Consider:

```
README.md
docs/architecture.md
docs/security.md
docs/api.md
ADR/
```

A coding agent that reads only source code may infer behavior incorrectly.

For example:

```
Source:
delete_document(id)
```

Documentation may reveal:

> Documents are soft-deleted because audit retention requires seven years of history.

Without the documentation, an agent might implement:

```sql
DELETE FROM documents WHERE id = ...
```

when the correct implementation is:

```sql
UPDATE documents
SET deleted_at = NOW()
WHERE id = ...
```

Documentation therefore provides **semantic context that source code alone may not reveal**.

---

12. Tests as Context

Tests are another critical source of context.

Suppose the agent sees:

```
def delete_document(document_id):
    ...
```

The implementation alone may not reveal whether deletion means:

- permanent deletion
- soft deletion
- asynchronous deletion
- deletion from search index
- deletion only for the current tenant

Tests may make the contract explicit:

```
def test_deleted_documents_are_not_searchable():
    ...
```

Tests therefore serve two roles:

```
Tests
 +-- verification
 `-- specification/context
```

This dual role is particularly important for coding agents.

The agent should often inspect tests **before modifying implementation**.

---

13. Task Decomposition

Large tasks create another context problem.

Consider:

> "Refactor the entire authentication subsystem."

Trying to reason about the entire problem simultaneously produces excessive context.

Instead, decompose:

```
Authentication refactor
+-- analyze current architecture
+-- define new interface
+-- migrate authentication state
+-- migrate authorization
```

```
+-- update API middleware
+-- update tests
`-- validate compatibility
```

Each subtask receives a narrower context.

This reduces:

$$C_{\text{task}}$$

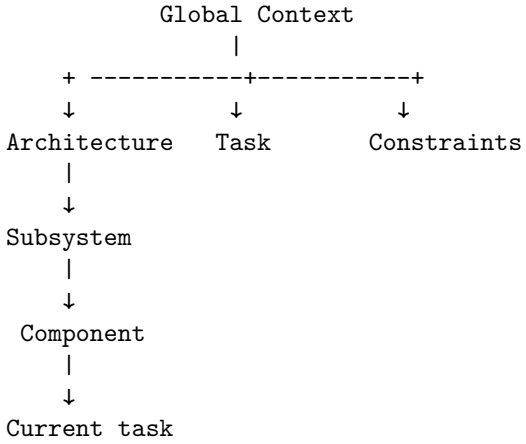
and often increases reasoning quality.

Task decomposition is therefore a **context-management technique**, not merely a project-management technique.

---

14. Context Hierarchies

A powerful architecture is hierarchical context.



For example:

Global

```
System architecture
Security model
Coding standards
```

Subsystem

```
Authentication architecture
```

Component

`OAuth service`

Task

`Fix redirect URI handling`

The model receives progressively more specific context as it moves down the hierarchy.

This is much more scalable than putting everything into one giant prompt.

---

15. Context Windows Are Not Infinite Memory

A common misconception is:

> "The model has a huge context window, so context management no longer matters."

This is incorrect.

Even very large context windows introduce problems:

Cost More tokens mean higher inference cost.

Latency Processing larger contexts takes longer.

Attention dilution Relevant information competes with irrelevant information.

Retrieval failure The important fact may be present but difficult to identify.

Instruction interference Large amounts of historical material may contain outdated or contradictory information.

State ambiguity The model may not know which version of a fact is authoritative.

Therefore:

$$\text{Large context capacity} \neq \text{unlimited useful context}$$

The relevant metric is not merely context size.

It is **context utility**.

---

16. Context Utility

We can define a conceptual utility function:

$$U(C \mid T) = \frac{\text{task-relevant information}}{\text{total information}}$$

This is intentionally simplistic, but useful.

Consider:

```
Context A:
100,000 tokens
10,000 relevant

Context B:
30,000 tokens
20,000 relevant
```

Context B may produce better results despite being much smaller.

Another formulation considers both information and cost:

$$U(C) = \frac{I(C;T)}{\text{Cost}(C)}$$

where:

- $I(C;T)$ represents useful information about task T
- $\text{Cost}(C)$ represents tokens, latency, or computational cost

The objective of context engineering is therefore not:

> maximize context.

It is:

> **maximize useful information per unit of context.**

---

17. Context Drift

Long-running agents can suffer from **context drift**.

Suppose the original task is:

```
Implement secure document retrieval.
```

After 50 iterations, the conversation becomes dominated by:

```
pytest failure
→ cache bug
→ dependency issue
→ formatting failure
→ retry
→ another test
→ unrelated warning
```

The agent can gradually lose focus on the original objective.

A context-management system should periodically re-anchor the agent:

```
Original objective:
Secure document retrieval.

Non-negotiable constraints:
- tenant isolation
- citations
- authentication

Current subtask:
Fix retrieval timeout.

Do not modify:
authentication architecture.
```

This creates a **stable task anchor**.

---

18. Stale Context

Context can also become stale.

Suppose the agent reads:

`src/auth.py`

and then modifies it.

An earlier copy of the file may remain in context.

Now the model sees:

```
Old version:
def authenticate(...)

New version:
def authenticate(...)
```

If the distinction is not explicit, the model may reason from obsolete information.

This creates a requirement:

> **Context systems must track freshness.**

Useful metadata includes:

```
artifact
version
timestamp
source
authority
```

For example:

```
src/auth.py
commit: a82f91
read: iteration 17
modified: iteration 21
```

The harness can then invalidate or refresh stale context.

---

19. Authority and Source Hierarchy

Not all context should have equal authority.

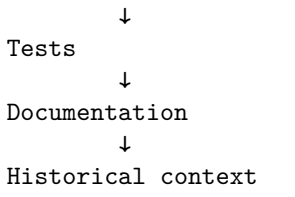
Consider:

```
ADR:
"All authentication must use OAuth."

Old README:
"Authentication uses API keys."

Source code:
currently contains API-key implementation.

User instruction:
"Migrate authentication to OAuth."
```

The agent must determine which information governs.

A useful hierarchy might be:

```
Explicit current task
        ↓
Security / system constraints
        ↓
Architecture decisions
        ↓
Current source
```



The exact hierarchy depends on the system.

The key principle is:

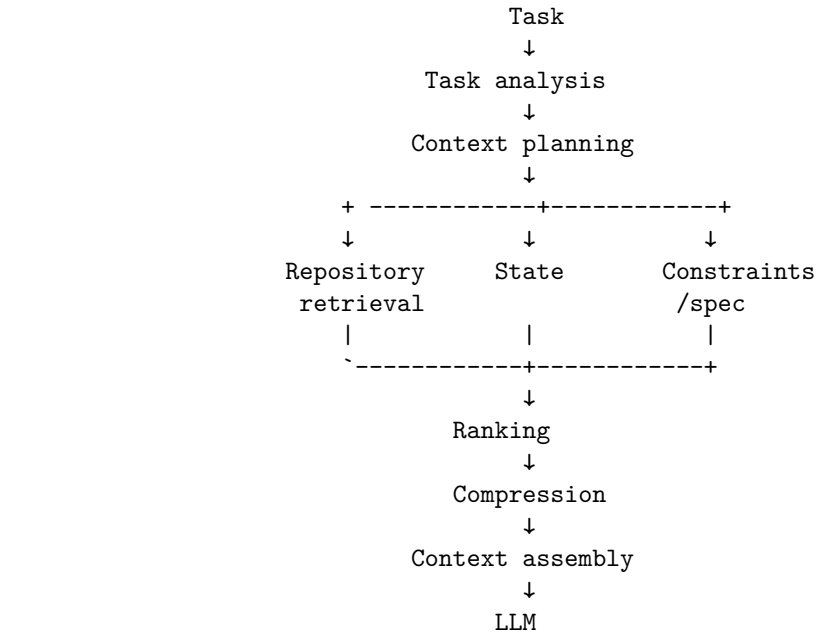
> **Context needs provenance and authority, not merely content.**

This becomes increasingly important as agents consume large amounts of heterogeneous information.

---

20. Context Construction as a Pipeline

A sophisticated context manager can be modeled as:



The important point is that **context construction itself is a computational pipeline**.

It can have:

- retrieval policies

- ranking algorithms
- token budgets
- caching
- summarization
- freshness tracking
- provenance
- prioritization
- task-specific policies

This is why context engineering increasingly resembles a conventional software subsystem.

---

21. The Agent's Context Is a Designed Artifact

In traditional programming, developers explicitly construct data structures.

In agentic systems, they also need to construct the model's **information environment**.

A context might look like:

```
SYSTEM
You are modifying a production Python service.

TASK
Add rate limiting to POST /query.

SPECIFICATION
...

ARCHITECTURE
FastAPI → QueryService → Redis → PostgreSQL

RELEVANT FILES
src/api/query.py
src/services/query.py
src/cache/redis.py

RELEVANT TESTS
tests/test_rate_limit.py

CONSTRAINTS
- 100 requests/minute/user
- HTTP 429 on violation
- internal calls bypass limit
```

```
CURRENT STATE
Implementation complete.
2 tests failing.

LATEST FEEDBACK
...
```

This is not merely a prompt.

It is a **constructed execution environment for reasoning**.

---

22. The Context Engineering Loop

Context itself should be iterative.

```
Task
 ↓
Retrieve context
 ↓
Agent reasons
 ↓
Agent discovers missing information
 ↓
Retrieve more context
 ↓
Agent reasons again
 ↓
Context becomes stale
 ↓
Refresh
 ↓
Compress
 ↓
Continue
```

This produces another feedback loop:

$$C_t \rightarrow Reasoning \rightarrow Information\ Need \rightarrow Retrieval \rightarrow C_{t+1}$$

The agent can therefore actively decide:

> "I don't have enough information to determine how authentication is configured."

and invoke:

```
search_code("authentication configuration")
```

This is a powerful pattern.

The agent is not merely consuming context.

It is **managing its own information acquisition** through tools.

---

23. Context and Agent Reliability

Why does context matter so much?

Because many agent failures are not fundamentally reasoning failures.

They are **information failures**.

Examples:

```
Wrong implementation
    ↓
Missing architectural constraint
```

```
Security vulnerability
    ↓
Security documentation omitted
```

```
Incorrect API
    ↓
Interface contract omitted
```

```
Repeated work
    ↓
Previous progress not preserved
```

```
Regression
    ↓
Relevant tests omitted
```

```
Wrong assumption
    ↓
Stale context
```

The model may be capable of solving the problem.

It simply did not receive the information required to solve it.

---

24. The Context Engineering Objective

We can formulate the problem more formally.

Given:

- task T
- repository R
- state S
- context budget B

construct:

$$C^* = \arg \max_{C:|C| \leq B} P(\text{successful outcome} \mid T, C, S)$$

This is the central optimization problem of context engineering.

The objective is not:

$$\max |C|$$

It is:

$$\max P(\text{success} \mid C)$$

subject to constraints on:

- tokens
- latency
- cost
- freshness
- relevance
- security

This formulation makes clear why context management is an engineering discipline.

---

25. Experiment — How Much Context Does an Agent Need?

Run the same coding task under five conditions.

Use the same:

- model
- repository
- task
- tools
- verifier
- execution environment

Only vary context.

Experiment A — Entire Repository

Give the agent everything.

```
All source
All tests
All documentation
All configuration
```

Experiment B — Relevant Files Only

Provide only the files believed to be relevant.

```
implementation
tests
direct dependencies
```

Experiment C — Architecture Summary

Provide:

```
architecture
module responsibilities
key interfaces
dependency relationships
```

but minimize source code.

Experiment D — Tests

Provide:

```
relevant implementation
relevant tests
```

Experiment E — Full Engineered Context

Provide:

```
architecture summary
relevant files
relevant tests
explicit constraints
acceptance criteria
current state
```

Now measure:

```
                 A       B       C       D       E
-----------------------------------------------------
Tests passed
```

```
Requirements met
Iterations
Tool calls
Tokens
Latency
Cost
Defects
Human corrections
```

The result will often reveal something surprising.

The best context may not be the largest context.

---

26. A Useful Context-Engineering Heuristic

For most coding tasks, prioritize information approximately in this order:

```
1. Current task/specification
2. Non-negotiable constraints
3. Relevant interfaces
4. Current state/progress
5. Relevant implementation
6. Relevant tests
7. Architectural context
8. Documentation
9. Historical context
10. Unrelated repository content
```

This is not a universal ordering.

The correct ordering depends on the task.

For debugging:

```
failure output
→ relevant code
→ tests
→ dependencies
→ architecture
```

For architecture work:

```
requirements
→ constraints
→ architecture
→ ADRs
→ interfaces
→ implementation
```

For security work:

```
security invariants
→ trust boundaries
→ authentication/authorization
→ data flows
→ implementation
→ tests
```

Context construction should therefore be **task-aware**.

---

27. Context Engineering as an Emerging Discipline

At this point, the implications become clear.

A modern AI engineer increasingly needs to understand:

```
What context does the model need?
Where does that context live?
How should it be retrieved?
How should it be ranked?
What should be summarized?
What must remain exact?
What is authoritative?
What is stale?
What should persist?
What should be discarded?
```

These are not purely prompt-writing questions.

They involve:

- information retrieval
- distributed systems
- state management
- caching
- databases
- software architecture
- knowledge representation
- evaluation
- security
- human-computer interaction

Context engineering therefore sits at the intersection of several disciplines.

---

28. From Prompt Engineering to Context Engineering

The progression can be viewed as:

```
Prompt Engineering
        ↓
"How should I phrase the request?"
        ↓
Context Engineering
        ↓
"What information should the agent have?"
        ↓
Agent Engineering
        ↓
"How should the agent act?"
        ↓
Specification Engineering
        ↓
"What exactly constitutes success?"
        ↓
Autonomous Engineering System
```

These disciplines are complementary.

A strong coding agent needs all of them.

---

29. The Deeper Insight: Context Is Part of the Program

In conventional software:

```
Program
+
Data
→
Behavior
```

In an AI agent:

```
Model
+
Context
+
Tools
+
State
→
Behavior
```

The context is therefore analogous to **runtime input to the program**.

Changing the context can change the behavior even when the model and code remain unchanged.

That means context should be:

- designed
- versioned
- tested
- evaluated
- monitored
- optimized

This is a major conceptual shift.

> **The context supplied to an agent is part of the system's executable behavior.**

---

30. Exercise — Build a Context Manager

Extend the coding agent from Chapter 15.

Instead of giving the model arbitrary repository information, build a context-management layer.

It should perform:

```
1. Parse the task
2. Identify relevant subsystems
3. Search the repository
4. Construct a repository map
5. Retrieve relevant files
6. Retrieve relevant tests
7. Retrieve constraints and documentation
8. Track current task state
9. Apply a token budget
10. Compress older information
11. Detect stale context
12. Construct the final model context
```

Instrument it.

Track:

```
context tokens
retrieval results
retrieval precision
files selected
files omitted
```

```
summaries generated
cache hits
context refreshes
task success
```

Then compare against a naive implementation.

The goal is not merely to make the prompt shorter.

The goal is to determine:

> **What information maximizes the probability that the agent reaches a correct, verified solution?**

That is the core problem of context engineering.

---

31. Key Takeaways

1. **Context is one of the primary resources of an AI agent.** Model capability alone is insufficient; the model must receive the right information.
2. **Context is not the same as state.** State represents what is true; context is the task-specific projection of that state presented to the model.
3. **Context should be budgeted.** The objective is not to maximize context size but to maximize useful information within a finite budget.
4. **More context does not necessarily produce better results.** Irrelevant information can dilute relevant information, increase cost, introduce contradictions, and make reasoning less reliable.
5. **Context selection is an information-retrieval problem.** Coding agents must retrieve relevant files, symbols, tests, documentation, architecture, and state.
6. **Repository maps provide structural context.** They allow an agent to understand the architecture of a codebase without reading every file.
7. **Context compression is state management.** Summaries should preserve decisions, constraints, assumptions, unresolved problems, and progress while discarding irrelevant history.
8. **Tests and documentation are context, not merely auxiliary artifacts.** Tests reveal behavioral contracts; documentation reveals semantic and architectural constraints.
9. **Task decomposition reduces context complexity.** Breaking a large problem into smaller tasks gives each reasoning step a more focused information environment.

10. **Context needs provenance and freshness.** The agent must know where information came from, how authoritative it is, and whether it is still current.

11. **Context should be hierarchical.** Global architecture, subsystem knowledge, component details, and task-specific information should be composed at different levels.

12. **The agent can actively manage its own context.** It can identify missing information, retrieve it through tools, refresh stale information, and compress older state.

13. **Context engineering can be formulated as an optimization problem.**

$$C^* = \arg \max_{C:|C|\leq B} P(\text{success} \mid T, C, S)$$

14. **Context is becoming part of the program.** In an AI system, changing the context can change system behavior even when the model and implementation remain unchanged.

15. **Context engineering is therefore a genuine software-engineering discipline.** It requires retrieval, ranking, compression, caching, state management, provenance, evaluation, and security—not merely better prompts.

The central lesson is:

> **A coding agent does not fail only because it cannot reason. It often fails because it was given the wrong information, too much information, stale information, or insufficient information.**

And that leads to a powerful design principle for agentic software:

$$\boxed{\text{Reliable Agent} = \text{Good Model} + \text{Good Specification} + \text{Good Context} + \text{Good Tools} + \text{Good Verification}}$$

The model provides the reasoning capability.

Context determines what that capability can actually reason about.